

The UTXO Model

How a ledger records who owns what as a pile of discrete, unspent coins — not as account balances — and why `hathor-core` is built this way.

SUBJECT hathor-core · Part I · Track B (domain concepts)

CHAPTER 07 · Foundations · Concepts

BRANCH study/hathor-core-studies

EDITION 2026 · v1

UTXO · Outputs & inputs · Locking & unlocking scripts · Change · Tokens · Authority outputs
· Account model (contrast)

Table of Contents

Chapter 7 — The UTXO Model	3
7.1 The intuitive model — and why it isn't the only one	3
7.2 The UTXO model: money as discrete coins	4
7.3 How a transaction moves money: consume and create	5
7.4 Why UTXO suits a distributed ledger	6
7.5 Ownership without an owner field: scripts	7
7.6 Hathor's spelling: inputs, outputs, tokens, authorities	7
Recap	8

Chapter 7 — The UTXO Model

What you'll learn in this chapter

- The two ways a ledger can record ownership — the **account model** and the **UTXO model** — and the trade-offs that lead a coin to choose one.
- What an **unspent transaction output (UTXO)** is, and why a balance is something you compute, not something stored.
- How a transaction **consumes** inputs and **creates** outputs, why inputs are spent whole (hence **change**), and the conservation rule that inputs must cover outputs.
- How **locking and unlocking scripts** decide who may spend an output (ownership without a stored "owner" field).
- Hathor's specifics: the TxInput / TxOutput classes, custom **tokens** riding the same model, and **authority outputs**.
- A **bridge** to where the model is implemented and indexed in the codebase.

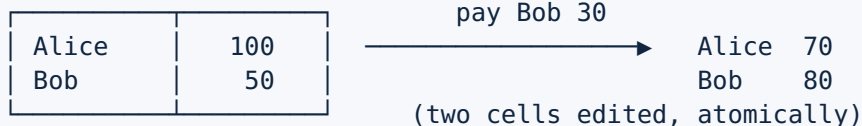
Chapter 6 left one question open: when the ledger says "who owns what," what is actually written down? There are two answers in wide use, and they lead to different code, different bugs, and different mental models. Hathor uses the one Bitcoin pioneered — the **UTXO model** — and you cannot read the transaction code, the verification rules, or the indexes without it. This chapter builds it from the account model you already intuitively hold, so the contrast does the teaching.

Track B may cite Hathor code, and this chapter does, lightly — but the model is general, so we develop it abstractly first and show Hathor's spelling at the end of each idea. The full code tour of the vertex classes is Chapter 25; here we want the concept solid.

7.1 The intuitive model — and why it isn't the only one

Ask anyone how a bank account works and they'll describe the **account model**¹: there is a row somewhere with your name and a number — your balance — and a payment subtracts from your number and adds to the recipient's. The ledger is a table of (account → balance), and a transaction is an edit to two cells.

ACCOUNT MODEL (e.g. a bank, or Ethereum)



This is natural and compact, and some major systems (Ethereum, most banks) use exactly it. But notice what it requires: a single mutable cell per account that everyone must update consistently, and an ordering — you must apply "Alice pays Bob" before "Bob pays Carol," or the balances go wrong. In a distributed setting with no central clock (Chapter 6), shared mutable cells and strict ordering are precisely the hard things. There is another way to record ownership that sidesteps both, and it is the one Hathi uses.

7.2 The UTXO model: money as discrete coins

The **UTXO model**² records ownership not as balances but as a collection of distinct, indivisible chunks of value, each created by some past transaction and not yet spent. UTXO stands for **Unspent Transaction Output**: an output of a transaction that no later transaction has consumed.

The mental shift is from a balance to a wallet full of coins of arbitrary denominations. You don't have "a balance of 70"; you have, say, a 50-coin and a 20-coin — specific objects, each traceable to the transaction that made it. Your balance is just what you get when you add up the coins you can spend:

Balance is derived, not stored. In a UTXO system there is no balance field anywhere. To know how much you have, you find every unspent output payable to you and sum their values. This is a defining property — and in Hathi it is literally true: there is no balance field on any vertex or address; balance is computed from unspent outputs (we cite this in §7.6).

A coin/output, conceptually, holds two things: **a value** (how much) and **a condition that says who may spend it** (the lock — §7.5). That's it. The ledger as a whole is the set of all outputs ever created, partitioned into spent and unspent. The unspent ones — the UTXOs — are the live money.

UTXO MODEL – Alice's "balance" is her unspent outputs

output A: value 50, locked to Alice	●	● = unspent (a UTXO) x = already spent
output B: value 20, locked to Alice	●	
output C: value 50, locked to Bob	●	
output D: value 30, locked to Alice	x	

Alice's balance = 50 + 20 = 70 (D doesn't count; it's spent)

7.3 How a transaction moves money: consume and create

A transaction in the UTXO model does exactly two things: it **consumes** some existing unspent outputs (as its inputs) and **creates** new outputs. An **input**³ is not a fresh piece of data — it is a pointer to an earlier output, saying "I am spending that specific coin." The outputs are the new coins, with new locks naming new owners.

Walk a concrete payment. Alice has the 50-output and the 20-output from §7.2 and wants to pay Bob 30:

TRANSACTION "Alice pays Bob 30"
INPUTS (coins consumed)

← output A (50)
← output B (20)

in: 70

OUTPUTS (coins created)

value 30 → locked to Bob	(Bob's new coin)
value 40 → locked to Alice	(Alice's CHANGE)

out: 30 + 40 = 70

Three rules fall out of this picture, and each one matters in the verification code (Ch 31):

1. Outputs are spent whole — hence change. An output is indivisible: you cannot spend "30 of a 50-coin." To pay 30, Alice must consume whole outputs summing to at least 30, then send the remainder back to herself as a second output. That returned remainder is **change**⁴ — exactly like getting coins back when you pay for a £7 item with a £10 note. This is why a simple payment usually has two outputs: one to the payee, one of change to the payer.

2. Inputs must cover outputs — conservation of value. The sum of input values must be at least the sum of output values; money cannot be created from nothing. (Any shortfall between inputs and outputs is, in many systems, a fee to the miner.) The one exception is the special transaction that mints new coins — in a blockchain, the block reward — which has outputs with no corresponding inputs, by protocol rule. In Hathon the genesis and block rewards are these minting points (Ch 9, Ch 22).

3. An output can be spent only once. The instant an output is used as some transaction's input, it ceases to be a UTXO. If two transactions both try to consume the same output, that is precisely the **double-spend** of Chapter 6, made concrete and local: two inputs pointing at one output. The whole consensus machinery (Ch 10, Ch 32) exists to ensure exactly one of them wins. This is the deep reason the UTXO model is attractive for a distributed ledger: a double-spend is not a subtle balance inconsistency spread across a table — it is a visible collision, two inputs naming the same prior output, easy to detect.

7.4 Why UTXO suits a distributed ledger

Now the trade-off, stated plainly, against the account model of §7.1.

What UTXO gives up: **simplicity and compactness**. Tracking a pile of discrete coins is more bookkeeping than editing one balance cell; wallets must select which coins to spend, manage change, and stitch a balance together from fragments. The account model is easier to think about for everyday "what's my balance."

What UTXO buys, and why a coin pays that price:

- **Double-spends are local and explicit.** As above: a conflict is two inputs citing one output — detectable by looking at the transactions themselves, with no global balance state to reconcile.
- **No shared mutable cell.** Outputs are created once and never modified — only consumed. There is no balance cell that every transaction races to update. Immutable-once-created data is far friendlier to a system where thousands of nodes apply transactions in slightly different orders (recall the appeal of immutability from Chapter 1 and the functional style from Chapter 2).
- **Parallelism and provability.** Independent transactions touch independent outputs, so they don't contend; and each coin carries its own provenance back through the outputs that made it.

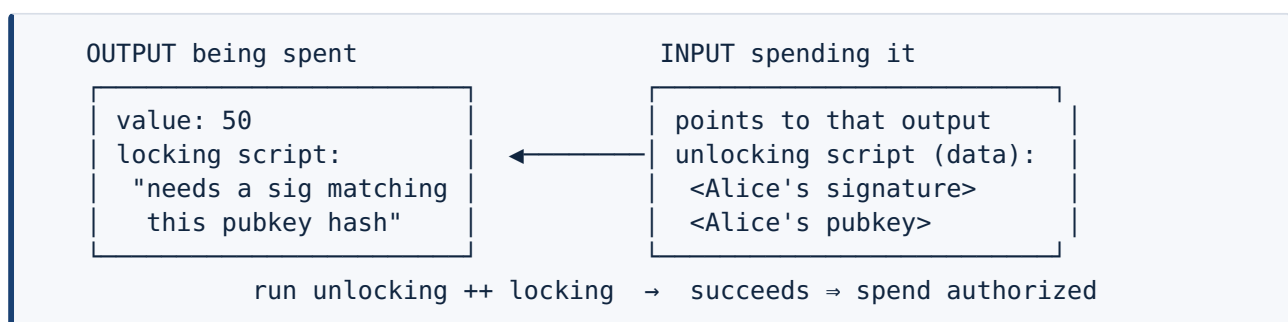
For a DAG-based ledger like Hathor's, where there isn't even a single linear order of transactions to lean on (Chapter 8), the "no shared mutable cell, conflicts are visible collisions" properties are decisive. UTXO and the DAG fit together; that is not a coincidence.

7.5 Ownership without an owner field: scripts

We keep saying an output is "locked to Alice." But there is no `owner = "Alice"` field. Ownership in the UTXO model is enforced by a small program, and this is one of the cleverest ideas in the design — worth meeting now because it reframes what "owning a coin" means.

Each output carries a **locking script**⁵ (also called a `scriptPubKey` in Bitcoin): a condition, expressed in a tiny stack-based scripting language, that must be satisfied to spend the output. The typical condition is "provide a digital signature⁷ from the private key behind this public-key hash." Each input that spends an output carries an **unlocking script**⁶ (the `data` field) supplying the missing pieces — typically the signature and public key.

To validate a spend, the node runs the unlocking script and the locking script together; if the combined program succeeds, the spend is authorized. So "Alice owns this coin" really means **"this output's locking script can only be satisfied by someone holding Alice's private key."** Ownership is the ability to produce a satisfying input, nothing more.



This indirection is what lets the same model express more than simple payments: multi-signature outputs ("needs 2 of these 3 signatures"), time locks ("not spendable until block height H"), and more — all by varying the locking script. The full script language and its evaluation are part of the vertex/verification story (Ch 25, Ch 31); here, hold the principle: **ownership is a satisfiable lock, not a name in a field.**

7.6 Hathor's spelling: inputs, outputs, tokens, authorities

Now the concrete code, briefly — the full tour is Chapter 25. In `hathor/transaction/base_transaction.py`, the two classes are exactly the model:

- **TxInput** (`base_transaction.py:936`) holds `tx_id` (the hash of the transaction whose output is being spent), `index` (which output, by position), and `data` (the unlocking script). That `(tx_id, index)` pair is the pointer-to-an-earlier-output from §7.3.

- **TxOutput** (`base_transaction.py:1022`) holds `value` , a `script` (the locking script of \$7.5), and `token_data` (next paragraph). No owner field, exactly as predicted.

Two Hathon-specific extensions are worth naming now so they're not a surprise later:

Custom tokens on the same model. Hathon is not single-currency. Besides the native token **HTR** (whose UID is the single byte `b'\x00'` , `conf/settings.py:29`), users create their own tokens, and they ride the same UTXO machinery. The output's `token_data` field encodes which token this output is denominated in (an index into the transaction's token list). So one output might be 50 HTR and another 1000 of some custom token, both unspent outputs in the same ledger. A token is created by a special transaction whose own hash becomes the token's UID (`TokenCreationTransaction` , Ch 25).

Authority outputs. Some outputs carry no spendable value but instead a permission — the right to mint more of a token, or to melt (destroy) it. These are **authority outputs**, flagged within the same `token_data` byte. They are how a token issuer retains control over supply, expressed in the very same output mechanism — a coin that grants a power rather than holding a value.

Finally, the derived-balance claim from §7.2, made real: the node maintains a **UTXO index** (`hathon/indexes/utxo_index.py`) keyed by (`address` , `token_uid`) , so that "what can this address spend" is answerable quickly. An output is recorded as spent not by deleting it but by noting, in metadata, which transaction spent it (`get_output_spent_by`) — which is what lets a voided transaction (Ch 10) cleanly reverse a spend and return the coin to the unspent set. We meet that index for real in Chapter 28.

Recap

Idea	Account model	UTXO model (Hathon)
Ownership recorded as	a mutable balance cell	a set of unspent outputs
Your balance	stored, read directly	computed by summing your UTXOs
A payment	edit two balance cells	consume inputs, create outputs
Spending part of a coin	adjust the number	impossible — spend whole, return change
A double-spend	a balance inconsistency	a visible collision: two inputs, one output
Who may spend	an <code>owner</code> field	a satisfiable locking script

Idea	Account model	UTXO model (Hathor)
Hathor classes	—	<code>TxInput</code> (:936), <code>TxOutput</code> (:1022)
Multiple currencies	—	<code>token_data</code> on each output; HTR = <code>b'\x00'</code>

The UTXO model records money as a heap of discrete, unspent, individually-locked outputs, where your balance is a sum you compute and a payment is the act of consuming whole coins and minting new ones with new locks. It costs more bookkeeping than account balances and buys, in return, exactly the properties a distributed ledger needs: outputs are immutable once created, there is no shared balance cell to race on, and a double-spend is a plainly visible collision rather than a hidden inconsistency. That last property is the hinge between this chapter and the next two: it is because spends are explicit pointers to prior outputs that Hathor can arrange transactions not in a single chain but in a graph — the **DAG** of Chapter 8 — and resolve the inevitable collisions by **voiding** (Chapter 10). With ownership now precisely defined, the next chapter rebuilds the structure of the ledger itself.

-
1. The account model (or account/balance model) records ownership as a mutable balance per account; a transaction debits one balance and credits another. Used by banks and by Ethereum. Contrast the UTXO model. ↩
 2. The UTXO model (Unspent Transaction Output) records ownership as a set of discrete outputs created by past transactions and not yet spent. A balance is the sum of one's unspent outputs. Used by Bitcoin and Hathor. ↩
 3. An input of a transaction is a reference to a specific earlier output that this transaction consumes (spends). In Hathor it is the `(tx_id, index)` pair plus an unlocking script. An input does not hold value itself; it points at the output that does. ↩
 4. Change is an output a transaction sends back to the payer, equal to the consumed inputs minus the amount paid (minus any fee). It exists because outputs must be spent whole, like receiving change after paying cash with a larger note. ↩
 5. A locking script (Bitcoin: `scriptPubKey`) is the spending condition attached to an output — a small program that must be satisfied to spend it, typically "provide a valid signature for this public key." It encodes ownership without naming an owner. ↩
 6. An unlocking script (Bitcoin: `scriptSig`) is the data an input supplies to satisfy the locking script of the output it spends — usually a signature and public key. In Hathor it is the input's `data` field. ↩
 7. A digital signature is data produced with a private key that anyone can verify with the matching public key, proving the signer authorized a specific message without revealing the private key. It is how a locking script confirms a spender's authority. (Full treatment with the crypto libraries in Ch 40.) ↩